Lesson 9 Sidebars

On the Scoreboard

VIDEO GAME CREATION

These aren't part of the lesson's required work. Think of them the way a textbook sidebar works: a deeper, tangential look at something this lesson's build actually ran into - why one approach was chosen over the others, what the alternatives were, or an idea worth a longer look than the Handout has room for. Pull one out for a discussion moment if it fits, hand it to a curious student, or skip it entirely - nothing here is required reading.

When the Fancy Tool Is the Wrong Tool

This week, five engines put a score on screen with a checkbox or a couple of code lines. Unity and Unreal needed a whole separate UI framework and several new concepts. Both are completely correct.

Scratch's built-in variable watchers - literally ticking a checkbox next to a variable - are the real, production-correct way to show a value on screen in Scratch. Not a shortcut, not a placeholder. The same is true of GameMaker's draw_gui and the equivalent lightweight text-drawing tools in Godot, GDevelop, and TIC-80: for a number or two on screen, these ARE the standard tool, not a simplified stand-in for a "real" one.

Unity and Unreal need genuinely more setup this week - Canvas/Text objects and prefabs in Unity, a full Widget Blueprint in Unreal - because they're general-purpose engines built for games far more complex than this course's capstone. Their real UI frameworks are correspondingly heavier because they're designed to also handle deeply nested menus, controller navigation, localization, and screens of every size and aspect ratio. That weight is genuinely earned on a shipping commercial game. It's also not avoidable here, because it's the one standard, correct tool those engines offer - there isn't a lighter "just show a number" option built in the way there is elsewhere.

001 Game Creator sits in between: it has its own dedicated, documented Interface/Field system - lighter than Unity or Unreal's frameworks, but still a purpose-built system rather than a one-line checkbox, with its own real concept to learn (an Interface's Fields can bind directly to a Variable and stay live-updating via an "Update Real-time" toggle).

The transferable lesson, worth stating directly: more setup is not the same thing as more correct. Recognizing when a lightweight built-in tool is actually the professionally right answer - rather than reaching for a bigger system just because it exists - is itself a production skill, and it applies well outside game engines too.

- **A bonus contrast worth raising in discussion:** This lesson's Unreal HUD updates itself only the instant Score or Lives actually changes (push-based - a UpdateHUD event is explicitly called from wherever the value changes). A more Update()-driven HUD instead checks every single frame whether anything changed, whether or not it did (poll-based). Both genuinely work, and this course uses both depending on the engine. Worth asking directly: for a value that changes constantly, versus one that changes rarely, which approach would you reach for, and why?